Delta Lake: Middle Level Guide for Rust Developers

Practical guide to work with Delta Lake using Rust. Architecture, operations, code examples, and best practices.

---

1. Architecture of Delta Lake

Delta Lake is an open-source storage format that adds ACID transactions on top of existing storage systems. Unlike plain Parquet files, Delta Lake keeps a transaction log that records every change to the table. This gives you safe concurrent reads and writes, time travel to old versions, and change auditing. The format was created by Databricks and is now a Linux Foundation project. The delta-rs library is a native Rust implementation. It does not need JVM and works much faster than the original Spark-based version.

The main parts of the architecture are: the _delta_log/ directory with JSON commit files, data files in Parquet format inside partition directories, checkpoint files in Parquet for faster loading, and the _last_checkpoint hint file. Each commit contains a set of actions: Protocol, Metadata, Add, Remove, CommitInfo and others. The delta-rs library implements the full protocol stack including log reading, snapshot management, and table operations.

2. Setting Up Your Rust Project

You need Rust version 1.91.1 or higher (edition 2024). The library is fully async and uses tokio as the runtime. Here is how to set up your project:

```
# Cargo.toml
[dependencies]
deltalake = { version = "0.32", features = ["datafusion", "rustls"] }
tokio = { version = "1", features = ["rt-multi-thread", "macros"] }
arrow = "58"

// main.rs
use deltalake::open_table;

[tokio::main]
async fn main() -> Result<(), Box<dyn std::error::Error>> {
  let table = open_table("./data/events").await?;
  println!("Table version: {}", table.version());
  let files: Vec<_> = table.get_file_uris()?.collect();
  println!("Total files: {}", files.len());
  Ok(())
}
```

Key feature flags: datafusion enables the SQL engine and DataFrame API; rustls (default) or native-tls for TLS connections to cloud storage; s3, azure, gcp for cloud backends. Only enable the features you need to make compilation faster and the binary smaller.

3. CRUD Operations with Tables

The delta-rs library gives you a full set of operations. All methods are called directly on the DeltaTable object. The old DeltaOps wrapper is deprecated.

3.1 Creating a Table

You create a table using the builder pattern. You define columns (StructField), partition columns, and the table name. The library automatically creates the _delta_log/ directory and writes the first commit:

```
use deltalake::kernel::{StructField, DataType, PrimitiveType};

let table = DeltaTable::new_in_memory()
  .create()
  .with_table_name("sensors")
  .with_columns(vec![
    StructField::new("id".into(),
      DataType::Primitive(PrimitiveType::Integer), false),
    StructField::new("value".into(),
      DataType::Primitive(PrimitiveType::Double), true),
    StructField::new("ts".into(),
      DataType::Primitive(PrimitiveType::Timestamp), true),
  ])
  .with_partition_columns(["year", "month"])
  .await?;
```

3.2 Writing Data

Data is written in Arrow RecordBatch format. You can choose the save mode: Append (add new data), Overwrite (replace all data), or MergeSchema. The library creates Parquet files, calculates column statistics, and writes a new commit to the transaction log:

```
use deltalake::protocol::SaveMode;

let schema = Arc::new(Schema::new(vec![
  Field::new("id", DataType::Int32, false),
  Field::new("value", DataType::Float64, true),
  Field::new("ts", DataType::Utf8, true),
]));

let batch = RecordBatch::try_new(schema, vec![...])?;

// Append new data
let table = table.write(vec![batch])
  .with_save_mode(SaveMode::Append)
  .await?;

// Or overwrite all data
let table = table.write(vec![batch])
  .with_save_mode(SaveMode::Overwrite)
  .await?;
```

3.3 Reading Data

Reading is done through scan_table() which returns a stream of RecordBatch. The library loads the table snapshot, gets the list of active files, filters out removed files, and reads only the needed partitions using predicate pushdown:

```
// Using scan API
let (_table, stream) = table.scan_table().await?;
let batches = collect_sendable_stream(stream).await?;
for batch in &batches {
  println!("Rows: {}", batch.num_rows());
}

// Or using DataFusion SQL
let ctx = SessionContext::new();
ctx.register_table("sensors", table.into())?;
let df = ctx.sql("SELECT * FROM sensors WHERE value > 50")
  .await?;
```

3.4 MERGE (Upsert)

MERGE lets you update, delete, and insert rows based on match conditions. This is a key operation for ETL and CDC pipelines. The syntax uses the builder pattern with when_matched_update, when_matched_delete, and when_not_matched_insert:

```
let merged = table.merge(source_df, "target.id = source.id")
  .when_matched_update(|update| {
    update.update("value", "source.value")
        .update("ts", "source.ts")
  })
  .when_not_matched_insert(|insert| insert.all())
  .await?;
```

4. Time Travel and Versioning

Every commit in Delta Lake gets a unique version number. You can load a table at any previous version or by timestamp. This is useful for debugging, reproducing results, auditing, and rolling back bad changes. The library supports three modes: Newest (default), Version(n), and Timestamp(dt). When loading by timestamp, it uses binary search over commits for fast lookup.

Method	Description	Example
load_version(n)	Load a specific version	table.load_version(5).await
load_with_datetime(dt)	Load by timestamp	table.load_with_datetime(dt).await
history(n)	Get commit history	table.history(10)
builder.with_version(n)	Version at open time	.with_version(5)
builder.with_datestring(s)	Timestamp at open time	.with_datestring("2025-01-01")

The history(n) method returns a list of CommitInfo objects. Each one has the commit time, operation type, user, and metrics. This is very useful for monitoring changes and understanding who did what and when.

5. Table Optimization

After many MERGE, UPDATE, and DELETE operations, a table accumulates many small files and tombstone files. This slows down reads. Delta Lake provides three key operations to keep tables fast.

5.1 OPTIMIZE (Compaction)

Compaction merges small files into larger ones, usually 128 MB to 1 GB. This greatly reduces the number of files and speeds up reads. Old files are marked as removed and physically deleted by VACUUM:

```
table.optimize().with_compaction().await?;
```

5.2 Z-ORDER

Z-ORDER groups data by specified columns inside files. This allows the engine to skip entire files when filtering (data skipping). The effect is very noticeable when you often query with the same filter columns:

```
table.optimize().with_z_order(["event_type", "region"]).await?;
```

5.3 VACUUM

VACUUM physically deletes files that are no longer used by the current snapshot. By default it removes files older than 7 days (retention period). Be careful: after VACUUM you cannot time travel to versions that referenced deleted files:

```
// Dry run first - check what will be deleted
table.vacuum().with_dry_run(true).await?;

// Then actually delete
table.vacuum()
  .with_retention_period(Duration::from_hours(168))
  .await?;
```

6. Schema Evolution

Delta Lake supports changing the table schema without rewriting all data. You can add new columns, change data types (with some limits), and rename columns. When reading old files, the library automatically fills null values for new columns that did not exist in those files. This lets you evolve the schema gradually, without any downtime.

```
table.add_constraint().with_constraint("id > 0").await?;
table.add_feature().with_feature("deletionVectors").await?;
```

7. Cloud Storage Backends

The delta-rs library supports three main cloud storage backends: Amazon S3, Azure Blob Storage, and Google Cloud Storage. Configuration is passed through a HashMap of string key-value pairs when opening a table.

Storage	Feature flag	Key parameters
Amazon S3	s3	AWS_ACCESS_KEY_ID, AWS_SECRET_ACCESS_KEY, AWS_REGION
Azure Blob	azure	AZURE_STORAGE_ACCOUNT_NAME, AZURE_STORAGE_ACCOUNT_KEY
Google GCS	gcp	GOOGLE_SERVICE_ACCOUNT, GOOGLE_PROJECT_ID
Local FS	(default)	Just a directory path

For S3, you can also enable commit-through-DynamoDB for strong consistency. This is important when multiple processes write to the same table at the same time. Without DynamoDB, the library uses optimistic locking with try-retry.

8. Best Practices

1. File size: Aim for files between 256 MB and 1 GB. Small files hurt performance, very large files make filtered reads slower.
2. Partitioning: Partition by columns with low cardinality (date, region). Too many partitions creates the small files problem.
3. Regular optimization: Run OPTIMIZE after every 5-10 MERGE or UPDATE operations to maintain good read performance.
4. VACUUM with care: Do not reduce retention below 7 days. Make sure time travel to old versions is no longer needed.
5. without_files(): For append-only apps, use .without_files() in DeltaTableBuilder. This saves a lot of memory.
6. Log buffer size: Tune the buffer for your storage. For cloud storage with rate limits, reduce the default value (4 x CPU cores).
7. Do not use DeltaOps: The old DeltaOps(table) API is deprecated. Use methods directly on DeltaTable.
8. Feature flags: Only enable what you need. Each extra feature makes compilation slower and the binary bigger.